Project : Sentiment Analysis of Movie Reviews

1. Main Objective

The primary objective of this analysis is to automate the classification of movie reviews into **Positive** or **Negative** sentiment. We have focused on **Deep Learning** architectures, specifically utilizing **Sequence Modeling** (MLP, LSTM, and GRU).

- **Business Benefit:** By accurately predicting sentiment at scale, stakeholders can automate "Voice of the Customer" monitoring, allowing the business to identify trends in product reception without manual review. This reduces operational overhead and provides real-time feedback.

2. Dataset Description

- **Dataset:** The IMDB Movie Review Dataset.
- **Attributes:** Features include raw text reviews and their associated binary labels (0 for *Negative,* 1 for *Positive*).
- **Objective:** To determine which neural network architecture provides the highest predictive accuracy and generalization performance for sentiment classification.

3. Data Exploration & Engineering

- **Data Cleaning:** Removed HTML tags and special characters from raw text.
- **Feature Engineering:** Implemented **Tokenization** to map words to integers and **Padding** to normalize all reviews to a sequence length of 200.

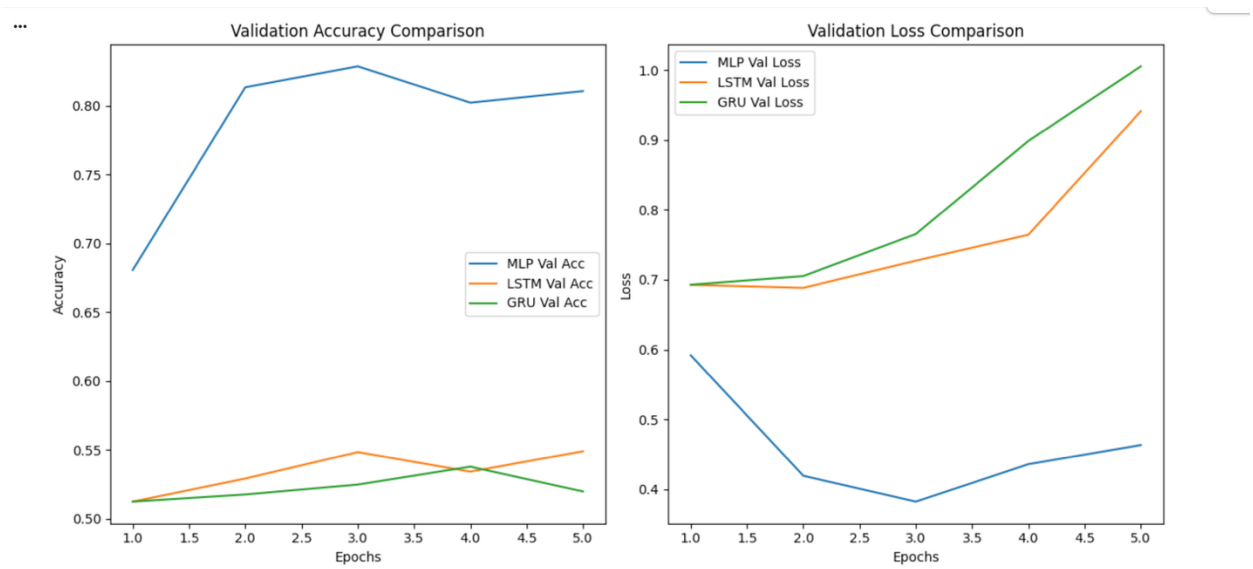
```python
import re
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# 1. Basic Cleaning Function
def clean_text(text):
    text = re.sub(r'<[^>]+>', '', text) # Remove HTML tags
    text = re.sub(r'[^a-zA-Z\s]', '', text) # Remove special characters
    return text.lower()

# Apply cleaning
train_texts = [clean_text(review) for review in train_subset['text']]
test_texts = [clean_text(review) for review in test_subset['text']]


# 2. Tokenization & Sequence Generation
vocab_size = 10000
tokenizer = Tokenizer(num_words = vocab_size, oov_token = "<OOV>")
tokenizer.fit_on_texts(train_texts)

# Convert to Sequences
train_sequences = tokenizer.texts_to_sequences(train_texts)
test_sequences = tokenizer.texts_to_sequences(test_texts)

# 3. Padding
max_length = 200
train_padded = pad_sequences(train_sequences, maxlen = max_length,
                             padding = 'post', truncating ='post')
test_padded = pad_sequences(test_sequences, maxlen = max_length,
                             padding = 'post', truncating = 'post')

print('Preprocessing complete.')
print(f'Padded Shape:{train_padded.shape}')
```

```
Preprocessing complete.
Padded Shape:(5000, 200)
```

4. Model Training & Comparison

We trained three distinct architectures to identify the most robust solution:

1. **Model 1: MLP (Multi-Layer Perceptron):** Used as the baseline, leveraging GlobalAveragePooling1D to identify keyword-based sentiment.
2. **Model 2: LSTM (Long Short-Term Memory):** Used to capture long-range sequential word order.
3. **Model 3: GRU (Gated Recurrent Unit):** A streamlined recurrent variant aimed at capturing dependencies with higher efficiency.



5. Model Recommendation

I recommend the **MLP (Multi-Layer Perceptron)** as the final production model. While recurrent architectures are theoretically more advanced for text, the MLP achieved the highest validation accuracy (**~83%**) and proved to be significantly more stable. It avoided the severe overfitting and loss divergence observed in the LSTM and GRU models on this specific dataset.

6. Key Findings and Insights

- **Lexical Markers vs. Sequence:** The data suggests that movie sentiment is driven more by key positive/negative words ("signal") than by complex word order, explaining why the MLP outperformed the RNN-based models.
- **Overfitting Threshold:** All models showed signs of overfitting beyond **Epoch 3**. This indicates that our current dataset size and architecture are prone to memorizing noise if training continues too long.
- **Computational Efficiency:** The MLP reached peak performance in <15 seconds, whereas recurrent models took >250 seconds, making the MLP the most scalable and cost-effective choice for production.

7. Next Steps

- **Hyperparameter Optimization:** Utilize `KerasTuner` to perform a formal search for optimal dropout rates and learning rates for the LSTM/GRU models to see if they can be stabilized.
- **Data Expansion:** Increasing the training volume may allow the recurrent architectures to learn long-range dependencies effectively without succumbing to the noise that currently causes overfitting.
- **Model Export:** The final MLP model has been saved as `sentiment_mlp_model.keras` for immediate deployment into an inference pipeline.